AIN4101: Machine Learning II

Lecture 3: RNNs, LSTM, and GRU for Sequential Data

Dr.Badrya Dahy

Badr University in Assiut (BUA)
School of Artificial Intelligence & Data Management

Week 3, Spring 2026

Outline

1. Sequential Data and Recurrent Networks
2. Training RNNs and Forecasting
3. Unstable Gradients
4. LSTM and GRU
5. Comparison and Practice
6. Summary

Learning Objectives - Today

By the end of this lecture, you will be able to:

1. Explain why sequential data needs models with memory.
2. Describe recurrent neurons, hidden states, and unrolling through time.
3. Compare sequence-to-vector, vector-to-sequence, sequence-to-sequence, and encoder–decoder structures.
4. Explain backpropagation through time (BPTT).
5. Prepare time-series data and build forecasting baselines.
6. Explain vanishing and exploding gradients and the standard remedies.
7. Describe the LSTM cell state and gates, and the GRU update and reset gates.
8. Compare RNN, LSTM, and GRU, and choose a model for a task.

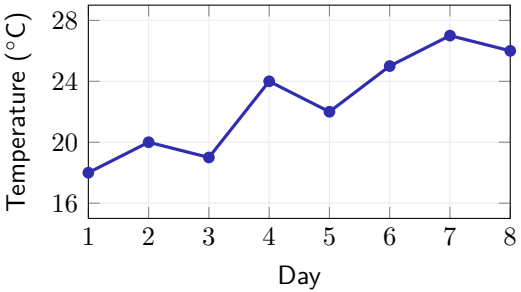
Key Takeaway

Lecture 2 turned tokens into vectors. Today we process *sequences* of vectors (or numbers) while keeping a memory of what came before.

Why Sequential Data?

Observations arranged in a meaningful order: earlier values influence later ones.

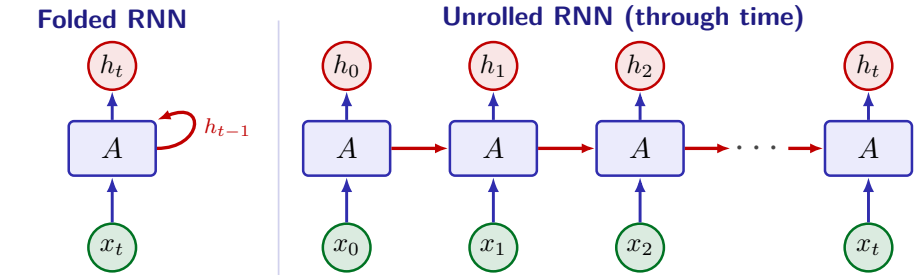
- **Time series:** temperature, prices, ridership.
- **Text:** words in a sentence.
- **Audio:** speech, music.
- **Other:** DNA, click streams, video frames.



Key Takeaway

Sequence length varies, order matters, and the past carries information about the future.

Recurrent Neuron and Layer



The same cell is reused

Same parameters at every time step

$$\mathbf{h}_t = \tanh(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b})$$

The same recurrent cell and weights are reused at every step; the hidden state carries information forward.

Memory Cells and the Hidden State

Memory cell

A part of a network that preserves a state across time steps: $\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$.

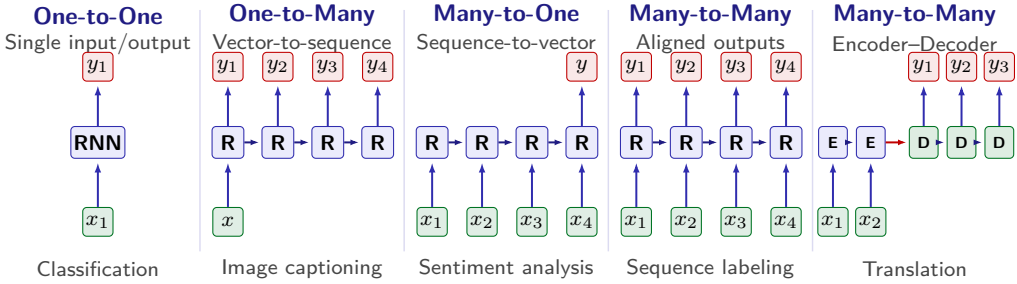
$$\mathbf{h}_1 \rightarrow \mathbf{h}_2 \rightarrow \mathbf{h}_3 \rightarrow \cdots \rightarrow \mathbf{h}_T$$

- $\mathbf{h}_t$ is a learned summary of $\mathbf{x}_1, \ldots, \mathbf{x}_t$ (short-term memory).
- In a simple RNN the state is also the output; in LSTM and GRU they can differ.

Example

"The weather was cold yesterday, but today it is warm." Interpreting *warm* depends on the order and context of earlier words.

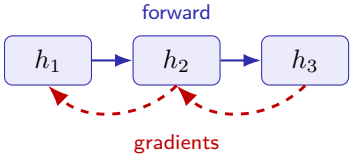
Input and Output Sequence Structures



Inputs in green, recurrent units in blue, outputs in red.

Backpropagation Through Time (BPTT)

1. **Unroll** the network over the sequence.
2. **Forward pass** through all steps.
3. Compute the loss $\mathcal{L} = \sum_t \mathcal{L}_t$ (or only $\mathcal{L}_T$ if only the last output has a target).
4. **Backward pass**: gradients flow back through every step.
5. **Sum** the gradients of the shared weights and update.



Important Note

Gradients pass through the same weights T times. Long sequences cost memory and can make gradients unstable; *truncated BPTT* limits how far back they flow.

Worked Example: One Recurrent Unit

Scalar unit: $h_t = \tanh(0.5\, x_t + 0.8\, h_{t-1})$, with $h_0 = 0$ and inputs $x = (1,\ 0.5,\ -1)$.

t	x_t	Pre-activation	h_t
1	1.0	$0.5 \cdot 1 + 0.8 \cdot 0 = 0.500$	0.462
2	0.5	$0.5 \cdot 0.5 + 0.8 \cdot 0.462 = 0.620$	0.551
3	-1.0	$0.5 \cdot (-1) + 0.8 \cdot 0.551 = -0.059$	-0.059

Key Takeaway

Without memory, the last input alone would give $\tanh(-0.5) \approx -0.462$. The state $h_3 \approx -0.059$ is pulled toward zero by the earlier positive inputs: the unit remembers the past.

Preparing Sequential Data: Sliding Windows

Sliding windows turn a series into input–target pairs. Series $[10, 12, 14, 16, 18, 20]$, window length 3:

Input window	Next value
$[10, 12, 14]$	16
$[12, 14, 16]$	18
$[14, 16, 18]$	20

Tensor shape (batch-first)

(batch, time, features)

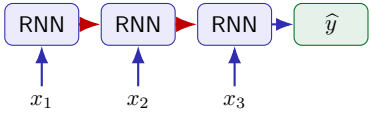
Example $(32, 20, 1)$: 32 sequences, 20 time steps, one feature per step.

Important Note

Split *chronologically* (train, validation, test in time order) and compute scaling statistics on the training part only, to avoid leaking the future.

Forecasting with a Simple RNN (Sequence-to-Vector)

$$\mathbf{h}_t = \tanh(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b})$$
$$\widehat{y} = \mathbf{w}_o^\top \mathbf{h}_T + b_o$$



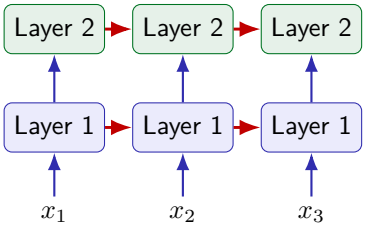
- Read the whole window, keep only the last state $\mathbf{h}_T$.
- A linear output layer maps it to one predicted value.
- A small single-layer RNN may fail to beat a good linear baseline.

A simple RNN should be compared against baseline forecasting models to evaluate its performance.

Example

Predict tomorrow's average temperature from the previous 20 daily temperatures.

Deep RNNs and Multivariate Series



A **deep RNN** stacks recurrent layers; layer 2 reads the state sequence of layer 1.

Multivariate data

Several features per step: temperature, humidity, pressure.

Important Note

More layers give more capacity but also more computation and a higher risk of overfitting.

Forecasting Several Time Steps Ahead

$$\underbrace{[20, 22, 23, 25, 26]}_{\text{history}} \longrightarrow \underbrace{[27, 28, 30]}_{\text{illustrative forecast}}$$

Recursive

Predict one step, feed it back as input, repeat.

Direct

One pass outputs all future values (vector output).

Seq-to-seq

Output at *every* step: many more training signals per sequence.

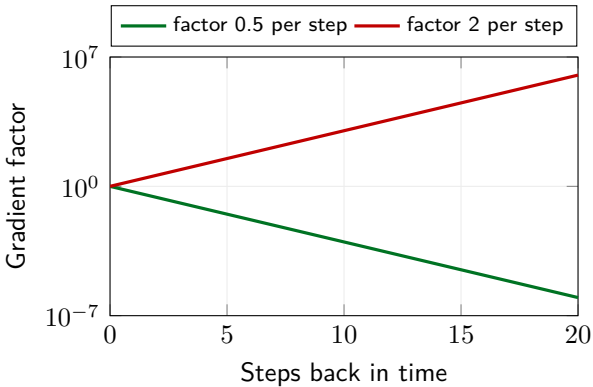
Important Note

Recursive forecasts accumulate error: a poor early prediction contaminates every later one.

Vanishing and Exploding Gradients

BPTT multiplies many per-step factors:

- Factors < 1: gradient vanishes. $0.5^{20} \approx 9.5 \times 10^{-7}$.
- Factors > 1: gradient explodes. $2^{20} = 1{,}048{,}576$.
- Vanishing $\Rightarrow$ no learning of long-range dependencies; exploding $\Rightarrow$ unstable training, NaN losses.



Scalar illustration only: real RNN gradients involve products of Jacobian matrices.

Handling Long Sequences

Technique	Purpose
Gradient clipping	Limit gradient size to stop exploding gradients
Layer normalization	Stabilize the activations at every time step
Dropout	Reduce overfitting
Truncated BPTT	Limit how many steps gradients flow back
LSTM	Keep relevant information in a gated cell state
GRU	Keep relevant information with a simpler gated state

Key Takeaway

Gating improves information and gradient flow, but it does not guarantee perfect memory for very long sequences.

The Short-Term Memory Problem

- In a simple RNN, information from early steps is repeatedly transformed and gradually lost.
- After many steps, the state keeps little trace of the first inputs.
- Example: a negation or key word at the start of a long review.

Two ideas behind LSTM and GRU

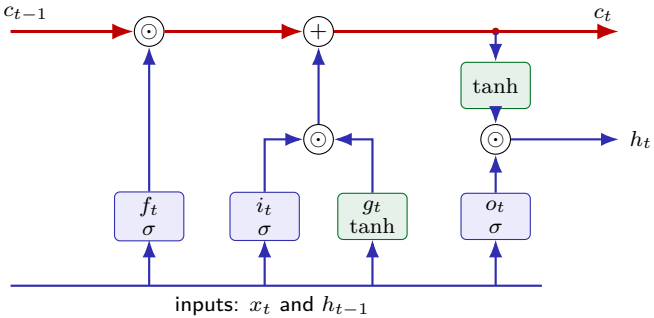
- A **protected memory path** that is updated gently, not rewritten at every step.
- Learned **gates**: values in $(0, 1)$ produced by a sigmoid, acting as soft on/off switches.

Key Takeaway

Gates let the network learn *what to keep, what to forget, and what to expose* at each step.

LSTM: Cell State and Three Gates

- $\mathbf{h}_t$: short-term state (also the output).
- $\mathbf{c}_t$: long-term cell state.
- **Forget gate** $\mathbf{f}_t$: what to erase from $\mathbf{c}_{t-1}$.
- **Input gate** $\mathbf{i}_t$: how much of the candidate $\mathbf{g}_t$ to add.
- **Output gate** $\mathbf{o}_t$: what part of the cell to expose.



Red path: cell state. Blue: sigmoid gates. Green: $\tanh$.

LSTM: Gate and State Equations

$$\mathbf{i}_t = \sigma(\mathbf{W}_{xi}\mathbf{x}_t + \mathbf{W}_{hi}\mathbf{h}_{t-1} + \mathbf{b}_i) \quad \text{input gate}$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_{xf}\mathbf{x}_t + \mathbf{W}_{hf}\mathbf{h}_{t-1} + \mathbf{b}_f) \quad \text{forget gate}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_{xo}\mathbf{x}_t + \mathbf{W}_{ho}\mathbf{h}_{t-1} + \mathbf{b}_o) \quad \text{output gate}$$

$$\mathbf{g}_t = \tanh(\mathbf{W}_{xg}\mathbf{x}_t + \mathbf{W}_{hg}\mathbf{h}_{t-1} + \mathbf{b}_g) \quad \text{candidate}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \quad \text{cell state}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{hidden state / output}$$

Important Note

σ outputs values in $[0, 1]$; $\odot$ is element-wise multiplication. The cell state is updated by *addition*, with no extra nonlinearity on the main path.

LSTM Step by Step

1. **Forget:** $\mathbf{f}_t \odot \mathbf{c}_{t-1}$ erases parts of the old memory.
2. **Write:** $\mathbf{i}_t \odot \mathbf{g}_t$ adds selected new information.
3. **Read:** $\mathbf{o}_t \odot \tanh(\mathbf{c}_t)$ exposes part of the memory as $\mathbf{h}_t$.

Example

Two memory cells with $\mathbf{c}_{t-1} = (2.0,\ -1.0)$, $\mathbf{f}_t = (0.9,\ 0.1)$, $\mathbf{i}_t = (0.5,\ 0.8)$, $\mathbf{g}_t = (0.4,\ 0.5)$.

$\mathbf{c}_t = (0.9 \cdot 2.0 + 0.5 \cdot 0.4,\ \ 0.1 \cdot (-1.0) + 0.8 \cdot 0.5) = (2.0,\ 0.3)$.

Cell 1 is kept ($f \approx 1$); cell 2 is mostly overwritten ($f \approx 0$).

Key Takeaway

The forget gate decides, per memory cell, between “keep” and “replace”.

GRU: A Simplified Gated Cell

Update gate $\mathbf{z}_t$

Balances *keeping* the old state against *using* the new candidate.

Reset gate $\mathbf{r}_t$

Controls how much of the old state is used to build the candidate.

- One state vector $\mathbf{h}_t$ (no separate cell state).
- No output gate: the whole state is the output.
- Two gates instead of three: fewer parameters.
- Introduced by Cho et al. (2014).

Key Takeaway

A GRU merges the LSTM's forget and input gates into one update gate and merges cell state and hidden state.

GRU: Gate and State Equations

$$\mathbf{z}_t = \sigma(\mathbf{W}_{xz}\mathbf{x}_t + \mathbf{W}_{hz}\mathbf{h}_{t-1} + \mathbf{b}_z) \quad \text{update gate}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_{xr}\mathbf{x}_t + \mathbf{W}_{hr}\mathbf{h}_{t-1} + \mathbf{b}_r) \quad \text{reset gate}$$

$$\mathbf{g}_t = \tanh(\mathbf{W}_{xg}\mathbf{x}_t + \mathbf{W}_{hg}\left(\mathbf{r}_t \odot \mathbf{h}_{t-1}\right) + \mathbf{b}_g) \quad \text{candidate}$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \mathbf{g}_t \quad \text{new state}$$

Example

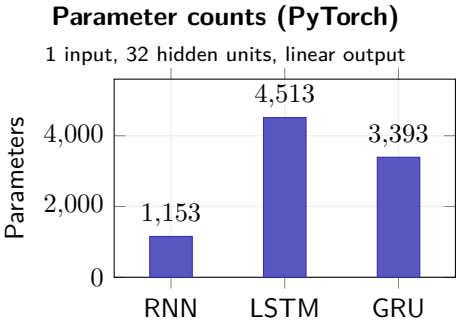
$z \approx 1$: keep the old state. $z \approx 0$: replace it with the candidate. $r \approx 0$: ignore the past when building the candidate.

Important Note

Conventions differ: PyTorch applies $\mathbf{r}_t$ after the matrix product and uses two bias vectors; some references swap $\mathbf{z}_t$ and $1 - \mathbf{z}_t$. The behavior is equivalent in spirit.

Comparison: RNN, LSTM, GRU

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Gates	0	3	2
Parameters	Lowest	Highest	Middle
Long dependencies	Hard	Better	Better
Complexity	Simple	Highest	Moderate



Key Takeaway

LSTM and GRU often perform similarly; GRU is lighter, LSTM has the extra cell state. Decide with validation results, not by default.

Choosing and Training a Sequence Model

1. Start with simple baselines: naive, seasonal naive, linear.
2. Scale inputs; split chronologically.
3. Try a small GRU or LSTM first; then add depth or width.
4. Use gradient clipping; consider layer normalization and dropout.
5. Prefer sequence-to-sequence training when forecasting.
6. Evaluate on a test set that comes *after* the training data in time.
7. For very long sequences, consider convolutions or attention.

Key Takeaway

Model choice is empirical: the best recurrent model is the one that wins on held-out data, at an acceptable cost.

Today's Key Concepts

1. Sequential data needs order-aware models with memory.
2. An RNN reuses one cell and carries a hidden state; BPTT trains it through time.
3. Forecasting: baselines first, sliding windows, chronological splits, direct vs. recursive vs. seq-to-seq.
4. Long sequences cause vanishing and exploding gradients; clipping and layer norm help.
5. LSTM: cell state plus forget, input, and output gates. GRU: one state plus update and reset gates.
6. Choose among RNN, LSTM, and GRU by validation results.

Key Takeaway

The progression is cumulative: feedforward $\rightarrow$ recurrence $\rightarrow$ BPTT $\rightarrow$ unstable gradients $\rightarrow$ gated memory (LSTM/GRU) $\rightarrow$ attention.

Lab 3 and Next Lecture

Lab 3 (hands-on code)

- Build sliding windows and chronological splits for a time series.
- Compute naive and linear baselines (MAE).
- Train a simple RNN, then a GRU and an LSTM; compare with the baselines.
- Add gradient clipping and layer normalization; try multi-step forecasting.

Next Lecture

Encoder–Decoder Models and Attention (building on Ch. 14–15).

Discussion

- Why can gradients vanish in a simple RNN, and how does the LSTM cell state help?
- What is the role of the GRU update gate?
- Why is a baseline comparison essential in forecasting?
- When would you prefer a GRU over an LSTM?

Questions?